Actividad Grupal – Clase 9: Construyan el Perceptrón Completo

Inteligencia Artificial · Ingeniería de Sistemas · Período 2026-02 · Docente: Jhon James Cano Sánchez · Lunes 19 de octubre de 2026

Campo	Detalle
Modalidad	Grupal. Los grupos se sortean al azar en clase (5 grupos de 4 y 1 grupo de 3).
Duración	90 minutos (incluye la introducción del docente)
Material	`Material_Actividad_Clase9.zip` en AVACO: `generar_datos.py`, `neurona_clase8.py`, `perceptron_reto.py` y la `Plantilla_Informe_Clase9.docx`
Entrega	**Un informe por grupo** (PDF) + el archivo `perceptron_reto.py` resuelto, en AVACO, **al finalizar la sesión**
Valoración	Hace parte de la nota de **talleres y prácticas** del segundo corte (nota máxima 5.0)

1. El Reto

Una cooperativa agrícola de la zona rural de Cartago quiere un modelo que, a partir de la **precipitación** y la **temperatura** de un lote, estime la **probabilidad** de que la cosecha sea exitosa, para priorizar la asistencia técnica. Tienen un archivo con 120 lotes (`cultivos_cartago.csv`, datos simulados con rangos realistas).

En la Clase anterior se construyó una neurona que aprendió la compuerta AND. **La pregunta de hoy es: ¿esa neurona sirve para este problema real? Y si no sirve, ¿qué le falta?** El grupo lo va a descubrir, lo va a construir y lo va a explicar en un informe.

> **La idea de la actividad:** Nadie les va a dar la solución. El archivo `perceptron_reto.py` trae el esqueleto del perceptrón con **7 retos sin resolver**. Cada reto tiene un **punto de control automático**: al ejecutar el script, este se detiene en el primer reto pendiente y muestra una pista. Su trabajo es investigar, discutir en grupo, resolver y, sobre todo, **entender por qué** cada pieza es necesaria.

2. Organización del Grupo

Rol	Responsabilidad
Coordinador(a)	Administra el tiempo, mantiene al grupo en el reto actual y es quien habla con el docente.
Programador(a) ("piloto")	Escribe el código en su computador. **Este rol rota en cada reto**: todos deben programar al menos una vez.
Verificador(a)	Lee la salida de cada ejecución, compara con el punto de control y revisa que el grupo entienda lo que escribió.
Relator(a)	Llena la plantilla del informe **mientras** el grupo avanza (no al final): hipótesis, decisiones, salidas y capturas.

En el grupo de 3 integrantes, el Coordinador(a) asume también el rol de Verificador(a).

Reglas de trabajo

- Se permite consultar las guías de clases anteriores, la documentación de NumPy y herramientas de IA. Pero **todo lo que se escriba debe poder explicarse**: el docente puede preguntarle a cualquier integrante, en cualquier momento, qué hace una línea del código. Si nadie la puede explicar, el reto no cuenta.
- No se comparte código entre grupos.

3. Preparación del Entorno (10 min)

Los comandos parten de la **carpeta del curso** (la que contiene `clase1`, ..., `clase8`); cada equipo la tiene en un lugar distinto, por eso no se usan rutas absolutas.

1. Abrir PowerShell y escribir `wsl`. Luego ir a la carpeta del curso (por ejemplo `cd ~` y después `cd IA`; usar el nombre propio). Verificar con `ls` que aparecen las carpetas de las clases.
2. Apagar el contenedor de la clase anterior si está encendido: `docker ps`; si aparece `python_ia`, ejecutar `docker compose down`.
3. Crear la carpeta de la clase y copiar el entorno: `mkdir -p clase9/src clase9/data clase9/notebooks` y luego `cp clase8/Dockerfile clase8/docker-compose.yml clase8/requirements.txt clase9/`.
4. Descargar de AVACO `Material_Actividad_Clase9.zip`, descomprimirlo y copiar los tres archivos `.py` dentro de `clase9/src` (desde el Explorador de Windows se puede llegar a la carpeta de WSL escribiendo `\\wsl$` en la barra de direcciones, o se pueden arrastrar los archivos a la carpeta `src` en VSCode).
5. Levantar el contenedor y abrir VSCode: `cd clase9`, `docker-compose up -d --build`, `code .`
6. Entrar al contenedor y a la carpeta de código: `docker exec -it python_ia bash` y luego `cd src`.
7. Generar los datos: `python generar_datos.py`. Debe indicar 120 filas y 65 lotes exitosos / 55 no exitosos.

4. Misión 0: Diagnóstico (10 min)

Antes de construir, hay que entender el problema. Ejecuten la neurona de la Clase 8, **sin modificarla**, con los datos reales:

```
python neurona_clase8.py
```

Abran también la imagen `diagnostico_clase8.png`. En grupo, respondan en el informe:

1. ¿La neurona llega a cero errores como en la compuerta AND? ¿Qué pasa con el número de errores de una época a otra?
2. ¿Qué ocurre con el tamaño de los pesos `w1` y `w2` a medida que avanzan las épocas? ¿Por qué creen que pasa?
3. Para el lote de 150 mm y 26 °C la neurona responde 1. ¿Qué tan segura está? ¿Le sirve esa respuesta a la cooperativa?
4. La neurona recorre los lotes uno por uno. ¿Qué pasaría con 1 millón de lotes?
5. **Hipótesis del grupo:** antes de abrir `perceptron_reto.py`, escriban una lista de lo que creen que le **falta** a esta neurona para funcionar bien con datos reales. Al final de la actividad compararán su lista con lo que realmente necesitaron.

> **Importante:** La hipótesis se califica por el razonamiento, no por acertar. Una hipótesis equivocada pero bien argumentada vale más que una lista copiada.

5. Misiones 1 a 7: Los Retos (35 min)

Abran `perceptron_reto.py` en VSCode, léanlo completo una vez en grupo (5 minutos) y ejecútenlo:

```
python perceptron_reto.py
```

El script se detendrá en el **RETO 1**. Cada reto está marcado en el código con `>>> RETO X <<<` y con la línea `ESCRIBA AQUÍ SU SOLUCIÓN`. Resuélvanlo, guarden con **Ctrl + S** y vuelvan a ejecutar, hasta que aparezca el mensaje final: *¡Todos los retos superados!*

Reto	¿Qué deben descubrir?	Dónde	Pregunta guía	El punto de control verifica…
1	Vectorización	`suma_ponderada`	La neurona de la Clase 8 recorría los lotes uno por uno con un ciclo `for`. ¿Cómo calcular `z` para los 120 lotes en una sola línea? (Clase 4: NumPy).	Con la neurona AND de la Clase 8 debe obtenerse `[-0.7 -0.2 -0.2 0.3]`.
2	Activación con probabilidad	`sigmoide`	La neurona de la Clase 8 respondía 0 o 1 sin decir qué tan segura estaba. ¿Qué función convierte cualquier número en un valor entre 0 y 1? ¿Por qué la función escalón no sirve para el descenso del gradiente?	Para z = −6, 0 y 6 debe dar ≈ 0.0025, 0.5 y 0.9975.
3	Medir el error	`calcular_perdida`	Contar errores no distingue entre "casi acierta" y "falló con total seguridad". ¿Cómo medir qué tan mal predice un modelo que entrega probabilidades?	Si el modelo dice 0.5 para todo, la pérdida es 0.6931; si acierta con 0.9, es 0.1054.
4A	Dirección del aprendizaje	`calcular_gradientes`	¿Hacia dónde y cuánto hay que mover cada peso para que la pérdida baje? Debe calcularse con todos los lotes a la vez, promediando.	Ejemplo pequeño con resultado conocido (lo muestra la pista).
4B	Paso de aprendizaje	Ciclo de entrenamiento	Con el gradiente ya calculado, ¿cómo se actualizan `w` y `b`? Compárelo con la regla de la Clase 8: ¿es la misma idea?	Verifica el primer paso del entrenamiento.
5	Preparación de las entradas	Antes del entrenamiento	Cuando el modelo ya entrene, observen los resultados: ¿son buenos? ¿Hay advertencias? ¿Qué diferencia hay entre los valores de la lluvia y los de la temperatura? (Clase 7).	Las entradas deben quedar con media 0 y desviación 1, y deben guardarse `media` y `desviacion`.
6	Frontera de decisión	Sección de gráficos	¿Dónde está exactamente la línea que separa "exitosa" de "no exitosa"? ¿Para qué valor de z la sigmoide da 0.5?	Los puntos calculados deben cumplir la ecuación de la frontera.
7	Predecir lotes nuevos	Final del archivo	Llegan 3 lotes que el modelo nunca vio. ¿Qué preparación necesitan antes de entrar al modelo y con qué valores?	Los lotes deben prepararse igual que los datos de entrenamiento.

> **Sobre el RETO 5:** Es el único que el script no les pide de inmediato: primero los dejará **entrenar** el modelo y ver los resultados. Analícenlos con cuidado antes de resolverlo. Anoten en el informe la exactitud y la pérdida **antes y después** de resolver el reto 5.

Qué registrar en el informe por cada reto

- **¿Qué faltaba?** Explicado con palabras propias, en 2 o 3 líneas.
- **¿Cómo lo descubrieron?** Qué probaron, qué consultaron, qué error vieron.
- **Código** con el que lo resolvieron (solo las líneas del reto).
- **Evidencia:** la línea `[OK] RETO X superado` o la salida relevante (captura o texto).

Ritmo sugerido

Minuto de la clase	El grupo debería ir en…
40	Misión 0 terminada e hipótesis escrita
52	Retos 1, 2 y 3 superados
65	Retos 4A, 4B y 5 superados (modelo entrenando bien)
75	Retos 6 y 7 superados
80	Experimento final registrado
90	Informe completo y entregado en AVACO

6. Experimento Final (5 min)

Con todos los retos superados, elijan **uno** de estos experimentos, ejecútenlo y registren el resultado:

- **A.** Cambiar `tasa_aprendizaje` a 0.001 y luego a 1.0 (volver después a 0.1). ¿Qué pasa con la pérdida final?
- **B.** Cambiar `epocas` a 20 y luego a 3000. ¿Vale la pena entrenar tanto?
- **C.** Agregar un cuarto lote a `lotes_nuevos` con valores elegidos por el grupo e interpretar la probabilidad.

7. El Informe (se construye durante la actividad; cierre: 10 min)

Usar la plantilla `Plantilla_Informe_Clase9.docx`. Estructura:

Sección	Contenido
Portada	Número de grupo, integrantes y rol de cada uno, fecha.
1. Diagnóstico	Respuestas de la Misión 0 e hipótesis del grupo (con la imagen `diagnostico_clase8.png`).
2. Bitácora de retos	Para cada reto: qué faltaba, cómo lo descubrieron, código y evidencia.
3. Resultados	Tabla con pérdida inicial y final, exactitud, w1, w2 y b; imágenes `curva_aprendizaje.png` y `frontera_decision.png`; predicciones de los 3 lotes nuevos.
4. Interpretación	¿Qué significa el signo de cada peso para un agricultor de Cartago? ¿Qué cambió con el reto 5 y por qué?
5. Experimento	El experimento elegido y su resultado.
6. Hipótesis vs realidad	Comparar la lista de la Misión 0 con lo que realmente faltaba. ¿Qué acertaron? ¿Qué no se imaginaban?
7. Conclusiones	Tres conclusiones del grupo y una pregunta que les quede abierta.
Anexo	El código completo de `perceptron_reto.py` resuelto y la salida final de la consola.

Forma de entrega

- Un integrante carga en AVACO, en la actividad "Actividad grupal Clase 9": el informe en PDF (`Informe_Clase9_GrupoX.pdf`) y el archivo `perceptron_reto.py` resuelto.
- Plazo: **al finalizar la sesión** (o el que indique el docente en clase).

8. Rúbrica (nota máxima 5.0)

Criterio	Peso	Nivel completo
Diagnóstico e hipótesis (Misión 0)	0.7	Respuestas basadas en lo observado en la salida y el gráfico; hipótesis argumentada.
Retos resueltos	1.5	Los 7 retos superados (`[OK]` en todos) y el código funciona.
Explicación de los retos	1.2	Cada reto explicado con palabras propias: qué faltaba y por qué es necesario.
Resultados e interpretación	0.8	Tabla de resultados, gráficos e interpretación de los pesos y del efecto del reto 5.
Experimento y "hipótesis vs realidad"	0.4	Experimento registrado y comparación honesta con la hipótesis inicial.
Informe y trabajo en equipo	0.4	Plantilla completa, código anexo, roles cumplidos y todos los integrantes pueden explicar el código.
Total	**5.0**	

> **Nota individual:** Si un integrante no puede explicar ninguna parte del código cuando el docente se lo pregunta, su nota puede ser diferente a la del grupo.